Exercises 2

donderdag 8 oktober 2026 12:26

```
postgres=# CREATE TABLE orders (
postgres(# id SERIAL PRIMARY KEY,
postgres(# customer_name TEXT,
postgres(# product_category TEXT,
postgres(# quantity INTEGER,
postgres(# price_per_unit NUMERIC
postgres(# ,orderd_data DATE,
postgres(# country TEXT
postgres(# );
CREATE TABLE
Time: 13.875 ms
```

```
postgres=# \COPY orders(customer_name, product_category, quantity, price_per_unit, orderd_data, c
ountry) FROM '/data/orders_1M.csv' DELIMITER ',' CSV HEADER;

COPY 1000000
Time: 7586.989 ms (00:07.587)
postgres=# FROM customer_name, product_category, price_per_unit
```

Activity 2.1:

A. Which order has the highest price_per_unit?

```
postgres=# SELECT *
FROM orders
ORDER BY price_per_unit DESC
LIMIT 5;
   id   | customer_name  | product_category | quantity | price_per_unit | orderd_data | country
--------+----------------+------------------+----------+----------------+-------------+---------
 841292 | Emma Brown     | Automotive       |        3 |         2000.0 | 2024-10-11  | Italy
 719717 | Leo Smith      | Automotive       |        3 |        1999.97 | 2024-01-19  | Sweden
 218300 | Amir Wilson    | Automotive       |        5 |        1999.96 | 2025-01-06  | USA
 861549 | Julia Thompson | Automotive       |        4 |        1999.95 | 2025-07-05  | Germany
 922147 | Julia Garcia   | Automotive       |        1 |        1999.95 | 2025-09-17  | China
(5 rows)
```

B. What are the top 3 product categories with the highest total quantity sold across all orders?

```
postgres=# SELECT product_category, SUM(quantity) AS total_quantity
postgres-# FROM orders
postgres-# GROUP BY product_category
postgres-# ORDER BY total_quantity DESC;
 product_category | total_quantity
------------------+----------------
 Health & Beauty  |         300842
 Electronics      |         300804
 Toys             |         300598
 Books            |         300448
 Fashion          |         300146
 Office Supplies  |         300093
 Grocery          |         299800
 Home & Garden    |         299668
 Automotive       |         299189
 Sports           |         298573
(10 rows)

Time: 641.775 ms
```

C. What is the total revenue per product category? (Revenue = price_per_unit × quantity)

```
postgres=# SELECT product_category, SUM(quantity * price_per_unit) AS total_revenue
FROM orders
GROUP BY product_category
ORDER BY total_revenue DESC;
 product_category | total_revenue
------------------+---------------
 Automotive       |  306589798.86
 Electronics      |  241525009.45
 Home & Garden    |   78023780.09
 Sports           |   61848990.83
 Health & Beauty  |   46599817.89
 Office Supplies  |   38276061.64
 Fashion          |   31566368.22
 Toys             |   23271039.02
 Grocery          |   15268355.66
 Books            |   12731976.04
(10 rows)

Time: 1106.404 ms (00:01.106)
```

D. **Who are the top 5 customers by total spending?**

```
postgres=# SELECT customer_name, SUM(price_per_unit) AS total_price
FROM orders
GROUP BY customer_name
ORDER BY total_price DESC
postgres-# LIMIT 10;
 customer_name  | total_price
----------------+-------------
 Carol Taylor   |   326009.91
 John Thomas    |   313324.54
 Alice Garcia   |   312368.21
 Daniel Young   |   310249.40
 Daniel Jackson |   310162.92
 Nina Lopez     |   309348.35
 Frank Johnson  |   309160.41
 Noah Moore     |   307993.42
 Alice Martinez |   307670.05
 Eve Taylor     |   307341.44
(10 rows)

Time: 737.815 ms
```

E. **Look at the spending totals in D — and at how many orders each of those customers has. What do you notice? Open ecommerce/dataset_generator.py and explain *why* the data looks like this.**

```
postgres=# SELECT customer_name, COUNT(*) AS number_of_orders, SUM(price_per_unit) AS total_price
 FROM orders
GROUP BY customer_name
ORDER BY total_price DESC
LIMIT 10;
 customer_name  | number_of_orders | total_price
----------------+------------------+-------------
 Carol Taylor   |             1028 |   326009.91
 John Thomas    |             1029 |   313324.54
 Alice Garcia   |              956 |   312368.21
 Daniel Young   |              973 |   310249.40
 Daniel Jackson |             1033 |   310162.92
 Nina Lopez     |              980 |   309348.35
 Frank Johnson  |              994 |   309160.41
 Noah Moore     |              975 |   307993.42
 Alice Martinez |             1009 |   307670.05
 Eve Taylor     |              999 |   307341.44
(10 rows)

Time: 817.805 ms
```

The order counts and spending totals are all very close together because the dataset is purely random. The generator doesn't contain real customers: for every order it picks a random first and last name, so each of the 1,056 possible name combinations has the same chance of being chosen. With 1,000,000 orders, every name ends up with roughly 947 orders.

The price of each order is also drawn at random, from the same distribution for every name, and is completely independent of who "placed" the order. Because each name has close to a thousand of these random orders, the highs and lows average out (the law of large numbers), and every total lands close to the same expected value.

That is also why more orders doesn't clearly mean more spending. The differences in order count are tiny (a few percent), while the random variation in price per order is much larger, so the price variation outweighs the effect of a few extra orders. For example, Daniel Jackson has the most orders in the top 10 but ranks only fifth. The differences we see are just noise, not real customer behaviour.

Activity 2.2:

Step 1:

rows in table	join result (COUNT(*))	time
50 000	2751822	4356.873ms (00:04.357)
100 000	109946508	15148.351ms (00:15.148)
200 000	439395606	101028.116ms (01:14.028)
1M	109383941260	2071483.292ms (34:31.483)

50 000:

```
postgres=# SELECT COUNT(*)
postgres-# FROM people_50k p1
postgres-# JOIN people_50k p2
postgres-# ON p1.country = p2.country
postgres-# ;
  count
----------
 27501822
(1 row)

Time: 4356.873 ms (00:04.357)
```

100 000:

```
postgres=# SELECT COUNT(*)
FROM people_100k p1
JOIN people_100k p2
ON p1.country = p2.country
;
   count
-----------
 109946508
(1 row)

Time: 15148.351 ms (00:15.148)
```

200 000:

```
postgres=# SELECT COUNT(*)
FROM people_200k p1
JOIN people_200k p2
ON p1.country = p2.country
;
   count
-----------
 439395606
(1 row)

Time: 101028.110 ms (01:41.028)
```

1M:

```
postgres=# SELECT COUNT(*)
FROM people_big p1
JOIN people_big p2
ON p1.country = p2.country;
    count
-------------
 10983941260
(1 row)

Time: 2071483.292 ms (34:31.483)
```

Step 2:

```
                                                
```

```
postgres=# SELECT COUNT(*)
FROM people_100k p1
JOIN people_100k p2
ON p1.country = p2.country
;
   count
-----------
 109946508
(1 row)

Time: 16834.651 ms (00:16.835)
postgres=# SELECT COUNT(*)
FROM people_100k p1
JOIN people_100k p2
ON p1.country = p2.country
;
   count
-----------
 109946508
(1 row)

Time: 16858.856 ms (00:16.859)
```

The index is used: the plan shows an Index Only Scan on idx_people_country with 0 heap fetches, so country is read straight from the index. But reading the data was never the bottleneck. Both scans together take less than 0.2 seconds of the 16-second total. Almost all the time goes to the join and the count. From 100,000 rows the join produces about 110 million combinations (n^2 per country), and that number is the same with or without an index.

Step 3:

```
postgres=# SELECT SUM(cnt * cnt) AS total_paris
postgres-# FROM (
postgres(# SELECT country, COUNT(*) AS cnt
postgres(# FROM people_100k
postgres(# WHERE country IS NOT NULL
postgres(# GROUP BY country) t;
 total_paris
-------------
   109946508
(1 row)

Time: 430.352 ms
```

```
postgres=# SELECT SUM(cnt * cnt) AS total_paris
FROM (
SELECT country, COUNT(*) AS cnt
FROM people_big
WHERE country IS NOT NULL
GROUP BY country) t;
 total_paris
-------------
 10983941260
(1 row)

Time: 5351.377 ms (00:05.351)
```

For both the `people_100k` and `people_big` datasets, we observe a significant difference in time, despite the number of results being the same.

Step 4:

1. **What the rewrite tells us about hardware and indexes:**
   The naive query joins every person to every other person in the same country. A country with n people produces n^2 rows, so that works grows quadratically whit the data. The rewrite computes the same number by reading the table once and keeping only one row per country. It finisher in millisecons. This show that the bottleneck was the algorithm, not the hardware or the access paht.
2. **If the business nedds the pairs themselves:**
   In the case the quadratic cost cannot be avoided by rewriting, because the output ietself has n^2 rows. Every system must at least produce and store or send those rows. For a few million people this quickly becomes billions of pairs and hunderds of gigabytes, regardless of the database.
   A bigger machine helps only linearly: twice the cores or I/O bandwidth gives at most about twice the speed, and in practice less, because writing the output to disk or over the network becomes the limit.

A cluster helps more, because this workload parallelises well. If the data is partitioned by country, each node can generate the pairs for its own countries independently, without exchanging data with other nodes. That gives a near-linear speedup with the number of nodes.

3. **Limits of a OLTP database for this workload:**
   PostgreSQL is an OLTP database: it is optimised for many small, fast transactions (insert an order, look up a customer), using a row-based storage layout on a single primary server. Heavy analytical queries like this one run into several limits:
   - **Single-node scaling:** parallel query is limited to the cores and memory of one machine. There is no built-in distributed join across servers, and vertical scaling in the cloud has a hard ceiling and becomes expensive quickly.
   - **Impact on other users:** a 35-second (or multi-hour) query consumes CPU, memory and I/O, and pushes useful data out of the buffer cache. This slows down the normal transactional workload that the database exists for.
   - **Cloud resource limits:** managed cloud databases often throttle storage IOPS and throughput. Large joins that spill to temporary files on disk then become even slower, and you pay for the provisioned capacity all the time.
   - **Row-based storage:** analytics often reads only a few columns over many rows, which columnar systems handle much more efficiently.